Java w IntelliJ IDEA - pakiety, wyjątki i kolekcje

BLOK 10 - import, enhanced for, try-catch, throw i struktury danych

1. Pakiet - porządkowanie klas

Gdy projekt składa się z wielu klas, nazwy mogą się powtarzać, a znalezienie właściwego pliku staje się trudniejsze. Java umożliwia grupowanie klas w **pakiety**. Na początku pliku można zapisać np. `package school;`, aby wskazać przynależność klasy do pakietu `school`. W standardowej organizacji projektu plik takiej klasy znajduje się w katalogu `school` wewnątrz katalogu źródłowego.

```
package school;

public class Student {
    public String name;
}
```

Pakiet stanowi część pełnej nazwy klasy. Klasa `school.Student` jest innym typem niż `store.Student`, mimo identycznej prostej nazwy `Student`. Dzięki temu możemy organizować większe projekty w logiczne grupy i ograniczać konflikty nazw.

2. Import - używanie klas z innych pakietów

Instrukcja `import` pozwala odwoływać się do klasy z innego pakietu za pomocą krótszej nazwy. Na przykład po `import java.util.Scanner;` możemy pisać `Scanner`, zamiast za każdym razem używać `java.util.Scanner`. Instrukcję importu umieszczamy po deklaracji pakietu, a przed definicją klasy.

```
import java.util.ArrayList;

ArrayList<String> names = new ArrayList<>();
names.add("Ala");
```

`import` nie kopiuje kodu klasy do naszego pliku. Jest informacją dla kompilatora, gdzie szukać używanej nazwy. Klasy z pakietu `java.lang`, np. `String`, są dostępne bez jawnego importu. Jeżeli w projekcie są dwie klasy o tej samej prostej nazwie, czasem trzeba użyć pełnej nazwy z pakietem.

3. Pętla for-each (enhanced for)

Jeżeli chcemy odczytać po kolei wszystkie elementy tablicy lub kolekcji, możemy użyć wygodnej pętli `for-each`. Nie musimy wtedy ręcznie zarządzać indeksem. W każdej iteracji zmienna podana po lewej stronie dwukropka otrzymuje kolejny element.

```
int[] scores = {10, 20, 30};
for (int score : scores) {
    System.out.println(score);
}
```

```
Wynik:
10
20
30
```

W pierwszej iteracji `score` wynosi 10, następnie 20 i 30. Pętla jest czytelna, gdy chcemy tylko odczytywać kolejne wartości. Gdy musimy znać indeks, np. żeby zmienić element `scores[i]`, zwykłe `for` z licznikiem bywa wygodniejsze. Zmiana samej lokalnej zmiennej `score` nie zmienia automatycznie elementu tablicy typu `int[]`.

4. Czym jest wyjątek?

Wyjątek opisuje nietypową sytuację podczas działania programu, na przykład próbę odczytania nieistniejącego pliku lub parsowania napisu, który nie jest liczbą. Zamiast zwracać normalny wynik, kod może zgłosić obiekt wyjątku. Jeżeli wyjątek nie zostanie odpowiednio obsłużony, zwykły przebieg programu zostaje przerwany i pojawia się komunikat błędu.

```
String value = "abc";
int number = Integer.parseInt(value);
System.out.println(number);
```

Tutaj `Integer.parseInt` nie potrafi odczytać liczby z tekstu `"abc"` i zgłasza `NumberFormatException`. Instrukcja wyświetlająca liczbę nie zostanie wtedy wykonana w normalnym przebiegu. Wyjątek może przekazać informację o problemie do miejsca, które potrafi go obsłużyć.

5. try-catch - przechwycenie wyjątku

Blok `try` zawiera kod, w którym może pojawić się wyjątek. Blok `catch` określa, co wykonać, jeżeli pojawi się wyjątek wskazanego typu. Jeżeli w `try` nie będzie takiego błędu, blok `catch` zostanie pominięty.

```
try {
    int number = Integer.parseInt("abc");
    System.out.println(number);
} catch (NumberFormatException exception) {
    System.out.println("Wpisz poprawną liczbę");
}
System.out.println("Program działa dalej");
```

```
Wynik:
Wpisz poprawną liczbę
Program działa dalej
```

`parseInt` zgłasza wyjątek, więc sterowanie przechodzi do `catch`. Komunikat przyjazny dla użytkownika zastępuje nieobsłużony błąd. Następnie program wykonuje instrukcję

znajdującą się za całym blokiem. Nie oznacza to, że każdą sytuację da się bezpiecznie zignorować - czasem program powinien poprosić o nowe dane albo przerwać operację.

6. Łapanie właściwego rodzaju wyjątku

Po nazwie klasy wyjątku poznajemy typ problemu. `NumberFormatException` dotyczy niepoprawnego tekstu liczbowego. `ArithmeticException` może pojawić się przy dzieleniu całkowitym przez zero. `ArrayIndexOutOfBoundsException` oznacza odwołanie do niewłaściwego indeksu tablicy. Lepiej obsługiwać konkretne wyjątki niż bez potrzeby przechwytywać wszystko.

```
int a = 10;
int b = 0;
try {
    System.out.println(a / b);
} catch (ArithmeticException e) {
    System.out.println("Nie wolno dzielić przez zero");
}
```

```
Wynik:
Nie wolno dzielić przez zero
```

Kiedy `b = 0`, operacja `a / b` dla liczb całkowitych zgłasza wyjątek. Program nie wyświetla nieistniejącego wyniku działania, tylko przechodzi do obsługi błędu. Zmiana `b` na 2 sprawi, że zostanie wypisana liczba 5, a blok `catch` nie zostanie uruchomiony.

7. finally - czynności wykonywane po try i catch

Blok `finally` jest przeznaczony na czynności, które powinny zostać wykonane przy opuszczaniu `try-catch`, na przykład porządkowanie zasobów. Zazwyczaj wykonuje się niezależnie od tego, czy w `try` pojawił się wyjątek. W pewnych sytuacjach, np. nagłego zakończenia procesu, wykonanie nie jest jednak gwarantowane.

```
try {
    System.out.println("Próba odczytu");
} finally {
    System.out.println("Koniec operacji");
}
```

```
Wynik:
Próba odczytu
Koniec operacji
```

W nowszym kodzie obsługującym strumienie często stosuje się `try-with-resources`, który automatycznie zamyka zasoby implementujące odpowiedni interfejs. `finally` nadal jest ważną konstrukcją i warto rozumieć, kiedy zostaje wykonany.

8. throw - świadome zgłoszenie wyjątku

Słowo `throw` służy do zgłoszenia konkretnego obiektu wyjątku. Używamy go np. wtedy, gdy metoda otrzyma wartość niezgodną z wymaganiami i nie powinna dalej wykonywać obliczeń. Pozwala to jasno wskazać, że przekazane dane są niepoprawne.

```
static void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Wiek nie może być ujemny");
    }
    System.out.println("Wiek: " + age);
}
```

Gdy wywołamy `setAge(-1)`, metoda od razu zgłosi `IllegalArgumentException`. Instrukcja `println` nie zostanie wykonana. Dla `setAge(20)` warunek jest fałszywy i wyświetlony zostanie prawidłowy wiek. Taki sposób kontrolowania danych jest często czytelniejszy niż zapisywanie niepoprawnych wartości i próba naprawiania ich później.

9. throws - informacja, że metoda może przekazać wyjątek

Słowo `throws` występuje w deklaracji metody i informuje, że może ona przekazać określony wyjątek do kodu wywołującego. Nie jest tym samym co `throw`, które faktycznie zgłasza obiekt wyjątku. Rozróżnienie tych dwóch słów jest szczególnie ważne przy operacjach plikowych.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

static String readFile(String filename) throws IOException {
    return Files.readString(Path.of(filename));
}
```

Metoda `readFile` może zakończyć się przekazaniem `IOException`, np. gdy plik nie zostanie odnaleziony. Kod wywołujący musi uwzględnić to ryzyko zgodnie z zasadami obsługi wyjątków sprawdzanych. Sama deklaracja `throws` nie gwarantuje, że błąd się pojawi; określa możliwość jego wystąpienia.

10. List i ArrayList - lista o zmiennej długości

Zwykła tablica ma długość ustaloną w chwili utworzenia. `ArrayList` jest strukturą danych, do której można dodawać kolejne elementy i usuwać istniejące. Dla listy korzystamy z metody `add`, a do pobierania elementu służy `get(index)`. Liczbę elementów odczytujemy przez `size()`.

```
import java.util.ArrayList;
ArrayList<String> names = new ArrayList<>();
names.add("Ala");
names.add("Ola");
System.out.println(names.size());
System.out.println(names.get(0));
```

```
Wynik:
2
Ala
```

W liście indeksy również zaczynają się od zera. `names.get(0)` odczytuje pierwszy element. Różnica względem zwykłej tablicy polega m.in. na składni: w tablicy używamy `array[0]`, a w `ArrayList` `list.get(0)`. Do liczby elementów odwołujemy się przez `size()`, a nie pole `length`.

11. Generics - dlaczego wpisujemy ArrayList<String>?

Zapis `ArrayList<String>` oznacza, że lista jest przeznaczona na obiekty typu `String`. Dzięki temu kompilator wykrywa próbę dodania wartości niepasującego typu i przy odczycie nie trzeba zgadywać, jaki rodzaj danych otrzymaliśmy. W nawiasach kątowych podajemy parametr typu, a nie pojedynczy element listy.

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(5);
numbers.add(10);
// numbers.add("tekst"); // błąd kompilacji
```

`Integer` jest klasą odpowiadającą prostemu typowi `int` w zastosowaniach obiektowych. Java często automatycznie zamienia `int` na `Integer` i odwrotnie w odpowiednim kontekście - nazywamy to boxing i unboxing. Do zapamiętania liczb w typowej liście używamy więc `ArrayList<Integer>`, a nie `ArrayList<int>`.

12. HashSet - element bez duplikatów

`HashSet` przechowuje unikalne elementy. Próba dodania wartości już obecnej w zbiorze nie zwiększa liczby elementów. Przydaje się na przykład do pamiętania unikalnych identyfikatorów, słów lub numerów. Nie należy polegać na kolejności iteracji `HashSet`.

```
import java.util.HashSet;
HashSet<String> cities = new HashSet<>();
cities.add("Łódź");
cities.add("Kraków");
cities.add("Łódź");
System.out.println(cities.size());
```

```
Wynik:
2
```

Trzecia operacja dodania nie zwiększa liczby elementów, ponieważ napis `"Łódź"` już występuje w zbiorze. Warto wiedzieć, że `HashSet` opiera się na regułach równości i haszowania elementów. Dla zwykłych napisów Java zapewnia ich odpowiednie działanie.

13. HashMap - wyszukiwanie wartości po kluczu

`HashMap` przechowuje pary klucz-wartość. Klucz pozwala odnaleźć powiązaną z nim wartość. Można na przykład zapisać nazwę produktu jako klucz i liczbę dostępnych sztuk jako wartość. Klucze w jednej mapie są unikalne; ponowne użycie tego samego klucza zastępuje przypisaną do niego wartość.

```
import java.util.HashMap;
HashMap<String, Integer> stock = new HashMap<>();
stock.put("długopis", 12);
stock.put("zeszyt", 4);
System.out.println(stock.get("długopis"));
```

```
Wynik:
12
```

`put` dodaje lub aktualizuje powiązanie, a `get` odczytuje wartość na podstawie klucza. Gdy podanego klucza nie ma w mapie, zwykłe `get` zwraca `null`. Warto więc sprawdzać obecność klucza za pomocą `containsKey` albo korzystać z `getOrDefault`, jeżeli chcemy mieć bezpieczną wartość domyślną.

14. ArrayDeque - kolejka i stos

`ArrayDeque` pozwala przechowywać elementy w strukturze, w której wygodnie dodaje się i usuwa elementy z początku lub końca. Dzięki temu nadaje się do realizowania kolejki FIFO (pierwszy dodany - pierwszy obsłużony) oraz stosu LIFO (ostatni dodany - pierwszy zdjęty).

```
import java.util.ArrayDeque;
ArrayDeque<String> queue = new ArrayDeque<>();
queue.addLast("Ania");
queue.addLast("Bartek");
System.out.println(queue.removeFirst());
System.out.println(queue.removeFirst());
```

```
Wynik:
Ania
Bartek
```

Osoby zostały dodane w kolejności Ania, Bartek i w tej samej kolejności są obsłużone, ponieważ zdejmujemy elementy z początku. Aby zachowywać się jak stos, moglibyśmy dodawać i usuwać elementy z tego samego końca. `ArrayDeque` nie przyjmuje elementów `null`.

Struktura	**Najważniejsza cecha**	**Gdy potrzebujemy...**
tablica	**stała długość**	**prostego zbioru wartości**
`ArrayList`	zmienna liczba elementów	listy z dodawaniem i usuwaniem
`HashSet`	brak duplikatów	unikalnych elementów
`HashMap`	klucz i wartość	wyszukiwania po kluczu

Struktura (cd.)	Najważniejsza cecha	Gdy potrzebujemy...
`ArrayDeque`	operacje na końcach	kolejki albo stosu

15. Kiedy wybrać tablicę, listę, zbiór lub mapę?

Jeżeli liczba elementów ma być stała i wystarcza prosta struktura jednego typu, dobrym wyborem jest tablica. Jeżeli chcemy łatwo dodawać i usuwać elementy przy zachowaniu kolejności listy, wybieramy `ArrayList`. Jeżeli najważniejsza jest unikalność elementów, przydaje się `HashSet`. Jeżeli szukamy wartości na podstawie klucza, używamy `HashMap`. Przy przetwarzaniu danych w kolejności ich napływu może przydać się `ArrayDeque`.

Nie ma jednej struktury najlepszej do wszystkich zadań. Zanim wybierzemy, określmy: czy ważna jest kolejność, czy można mieć duplikaty, czy potrzebujemy wyszukiwania po kluczu i czy liczba elementów się zmienia. Odpowiedzi na te pytania zwykle wskazują odpowiednią klasę.

Przykład 1. PackageImportDemo

Łączymy pojęcie pakietu i importu z rzeczywistą deklaracją klasy.

```
package block10;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class PackageImportDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Ada");
        System.out.println(LocalDate.now());
        System.out.println(names);
    }
}
```

- W zapisie `List<String> names = new ArrayList<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(LocalDate.now());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(names);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Example01EnhancedFor

Sumujemy elementy tablicy bez ręcznego wyliczania indeksów.

```
package block10;

public class Example01EnhancedFor {
    public static void main(String[] args) {
        int[] numbers = {2, 4, 6, 8};
        int sum = 0;

        for (int number : numbers) {
            sum += number;
        }

        System.out.println(sum);
    }
}
```

Wynik:

```
20
```

- Instrukcja `int[] numbers = {2, 4, 6, 8};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Zapis `int sum = 0;` tworzy zmienną `sum` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Pętla `for (int number : numbers) {` pobiera po kolei elementy wskazanej tablicy lub kolekcji. W każdej iteracji zmienna przed dwukropkiem oznacza następny element.
- Instrukcja `sum += number;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.
- Instrukcja `System.out.println(sum);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 3. Example02TryCatch

Program przechwytuje wybrany wyjątek i może przejść do kolejnych instrukcji.

```
package block10;

public class Example02TryCatch {
    public static void main(String[] args) {
        String text = "abc";
        try {
            int number = Integer.parseInt(text);
            System.out.println(number);
        } catch (NumberFormatException e) {
            System.out.println("Niepoprawna liczba: " + text);
        }
    }
}
```

Wynik:

```
Niepoprawna liczba: abc
```

- Zapis `String text = "abc";` tworzy zmienną `text` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

- W bloku `try {` umieszczamy operację, która może zakończyć się wyjątkiem; wykonywanie może przejść do odpowiadającego `catch`.
- Zapis `int number = Integer.parseInt(text);` tworzy zmienną `number` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(number);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Niepoprawna liczba: " + text);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example03Finally

Sprawdzamy, kiedy wykonuje się blok `finally`.

```
package block10;

public class Example03Finally {
    public static void main(String[] args) {
        try {
            int result = 10 / 2;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Błąd dzielenia");
        } finally {
            System.out.println("Koniec sekcji try/catch");
        }
    }
}
```

Wynik:

```
5
Koniec sekcji try/catch
```

- W bloku `try {` umieszczamy operację, która może zakończyć się wyjątkiem; wykonywanie może przejść do odpowiadającego `catch`.
- Zapis `int result = 10 / 2;` tworzy zmienną `result` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(result);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Błąd dzielenia");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Koniec sekcji try/catch");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 5. Example04Throw

Badamy, kiedy własny kod świadomie zgłasza wyjątek za pomocą `throw`.

```
package block10;

public class Example04Throw {
    static void validateAge(int age) {
        if (age < 0 || age > 130) {
            throw new IllegalArgumentException("Wiek poza zakresem: " + age);
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(150);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Wynik:

```
Wiek poza zakresem: 150
```

- W zapisie `if (age < 0 || age > 130) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Wiersz `throw new IllegalArgumentException("Wiek poza zakresem: " + age);` świadomie zgłasza wyjątek, sygnalizując problem zamiast kontynuować zwykłe wykonanie.
- W bloku `try {` umieszczamy operację, która może zakończyć się wyjątkiem; wykonywanie może przejść do odpowiadającego `catch`.
- Instrukcja `System.out.println(e.getMessage());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 6. Example05ArrayList

Tworzymy listę, dodajemy imiona, usuwamy jedno i wyświetlamy pozostałe.

```
package block10;

import java.util.ArrayList;
import java.util.List;

public class Example05ArrayList {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Ala");
        names.add("Ola");
        names.add("Jan");
        names.remove("Ola");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Wynik:

```
Ala
Jan
```

- W zapisie `List<String> names = new ArrayList<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Pętla `for (String name : names) {` pobiera po kolei elementy wskazanej tablicy lub kolekcji. W każdej iteracji zmienna przed dwukropkiem oznacza następny element.
- Instrukcja `System.out.println(name);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 7. Example06HashSet

Dodajemy elementy do zbioru, w którym nie powstają duplikaty.

```
package block10;

import java.util.HashSet;
import java.util.Set;

public class Example06HashSet {
    public static void main(String[] args) {
        Set<String> tags = new HashSet<>();
        tags.add("java");
        tags.add("algorytmy");
        tags.add("java");
        System.out.println(tags);
    }
}
```

Przykładowy wynik (kolejność elementów może być inna):

```
[algorytmy, java]
```

- W zapisie `Set<String> tags = new HashSet<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(tags);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 8. Example07HashMap

Przechowujemy wartości pod kluczami i odczytujemy je na podstawie nazwy klucza.

```
package block10;

import java.util.HashMap;
import java.util.Map;

public class Example07HashMap {
    public static void main(String[] args) {
        Map<String, Integer> points = new HashMap<>();
        points.put("Anna", 10);
        points.put("Jan", 7);
        points.put("Anna", 12);

        System.out.println(points.get("Anna"));
        System.out.println(points);
    }
}
```

Przykładowy wynik (kolejność par może być inna):

```
12
{Jan=7, Anna=12}
```

- W zapisie `Map<String, Integer> points = new HashMap<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(points.get("Anna"));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(points);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 9. Example08ArrayDeque

Badamy, jak kolejność dodawania i usuwania wpływa na działanie kolejki lub stosu.

```
package block10;

import java.util.ArrayDeque;
import java.util.Deque;

public class Example08ArrayDeque {
    public static void main(String[] args) {
        Deque<String> queue = new ArrayDeque<>();
        queue.addLast("pierwszy");
        queue.addLast("drugi");
        System.out.println(queue.removeFirst());

        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(10);
        stack.push(20);
        System.out.println(stack.pop());
    }
}
```

Wynik:

```
pierwszy
20
```

- W zapisie `Deque<String> queue = new ArrayDeque<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(queue.removeFirst());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- W zapisie `Deque<Integer> stack = new ArrayDeque<>();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(stack.pop());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 10. SafeDivision

Przy dzieleniu sprawdzamy niebezpieczny przypadek dzielnika równego zero.

Polecenie: Obsłuż NumberFormatException i ArithmeticException.

```
package block10;

public class SafeDivision {
    public static void main(String[] args) {
        String aText = "20";
        String bText = "0";
        // TODO
    }
}
```

- Zapis `String aText = "20";` tworzy zmienną `aText` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `String bText = "0";` tworzy zmienną `bText` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

16. Zadania - wyjątki, importy i kolekcje

Zadanie 1. Utwórz pakiet `school` i klasę `Student`. Napisz drugą klasę w innym pakiecie i zaimportuj `school.Student`. Wyjaśnij, po co w tym miejscu potrzebny jest import.

Zadanie 2. Wczytaj liczbę podaną jako tekst. Jeśli tekst jest niepoprawny, wypisz komunikat „Niepoprawna liczba” zamiast kończyć program nieobsłużonym wyjątkiem.

Zadanie 3. Utwórz metodę `divide(int a, int b)`, która nie wykona dzielenia przez zero. Wybierz, czy metoda ma zgłosić wyjątek, czy zwrócić informację o błędnych danych. Uzasadnij wybrany sposób.

Zadanie 4. Dodaj do `ArrayList<String>` pięć imion. Usuń drugie, wyświetl liczbę elementów i przejdź przez pozostałe wartości za pomocą enhanced for.

Zadanie 5. Do `HashSet<Integer>` dodaj liczby 3, 5, 3, 7, 5. Jaka będzie liczba elementów i dlaczego nie ma sensu zakładać konkretnej kolejności ich wypisywania?

Zadanie 6. Zbuduj `HashMap<String, Integer>` przechowującą liczbę punktów trzech graczy. Pobierz punkty jednego gracza i zmień je przez ponowne `put` z tym samym kluczem.